

Black box Unit Testing

Dileepa Fernando

Learning Outcomes

By the end of this session, you should be able to:

1. Differentiate specification-based vs code-based testing
2. List testing levels based on design abstraction
3. Apply boundary value testing
4. Apply equivalence class testing
5. Apply decision table testing

Testing Approaches

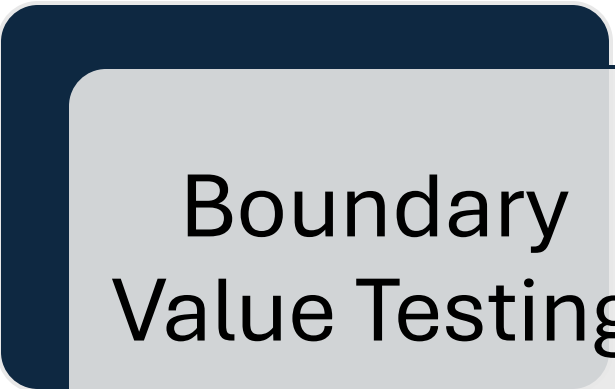
- Specification-based (Black-box)
 - Treats function as mathematical mapping
 - Test cases based solely on **input/output** spec
 - Focuses on confirming requirements
- Code-based (White-box)
 - Tests derived from knowledge of implementation
 - Seeks to maximize code coverage

Testing Levels

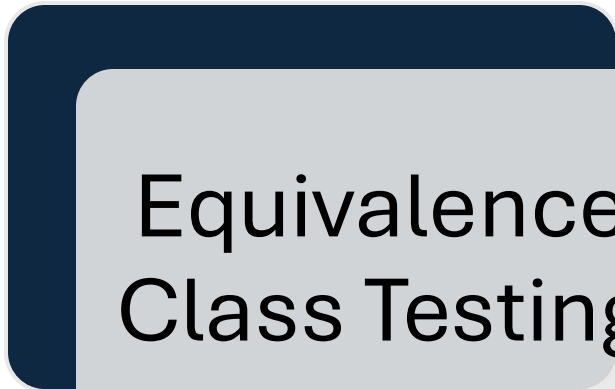
Design Abstraction	Testing Level
Requirements/Spec	System testing
High-level design	Integration testing
Detailed design	Unit testing



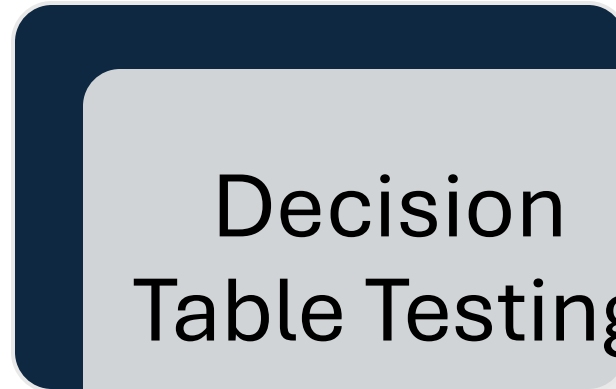
Specification-based Unit Testing Techniques



Boundary
Value Testing



Equivalence
Class Testing



Decision
Table Testing



Boundary Value Testing

Input
value ranges
are key

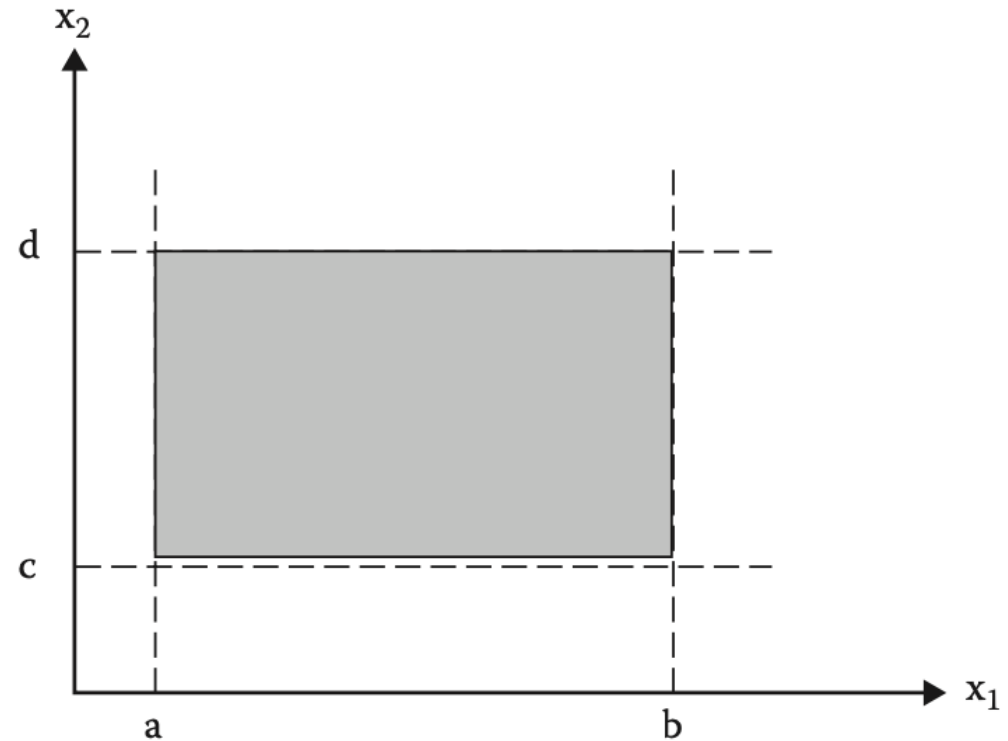
Faults often
occur at
limits

Normal Boundary Value Testing

For function $f(x_1, x_2)$ with domains $[a, b]$, $[c, d]$, test combinations covering:

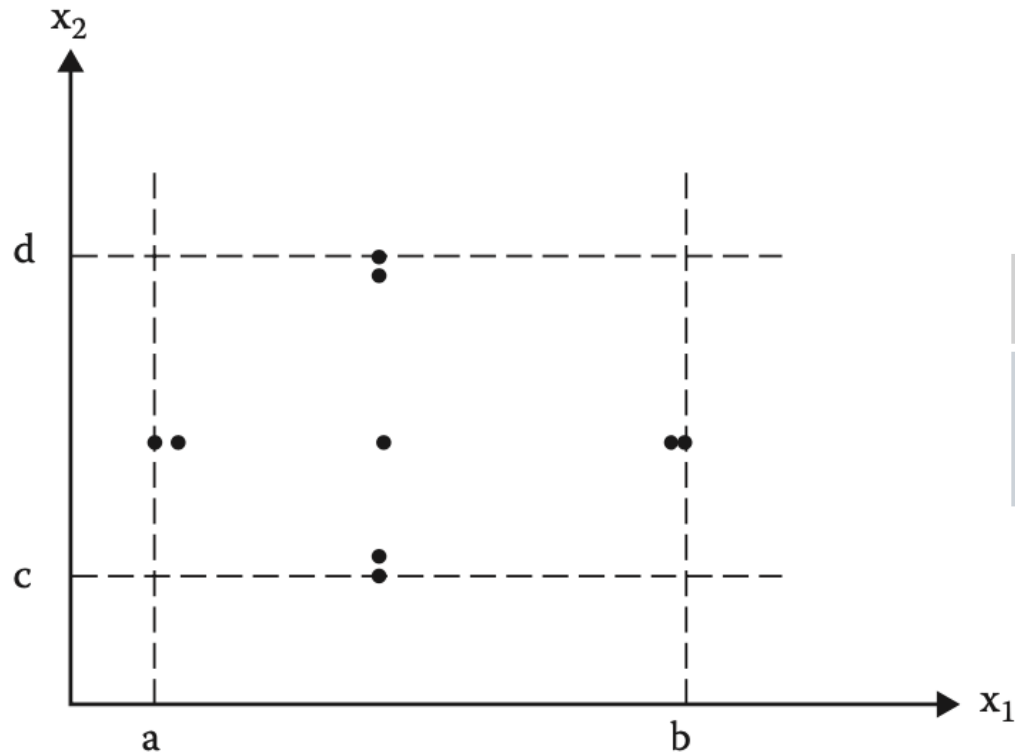
Assumptions:

1. assuming only valid inputs and valid type
2. assuming error is caused by one variable's boundary value



Normal Boundary Value Testing

For function $f(x_1, x_2)$ with domains $[a,b]$, $[c,d]$,
test combinations covering:



(mid, mid)



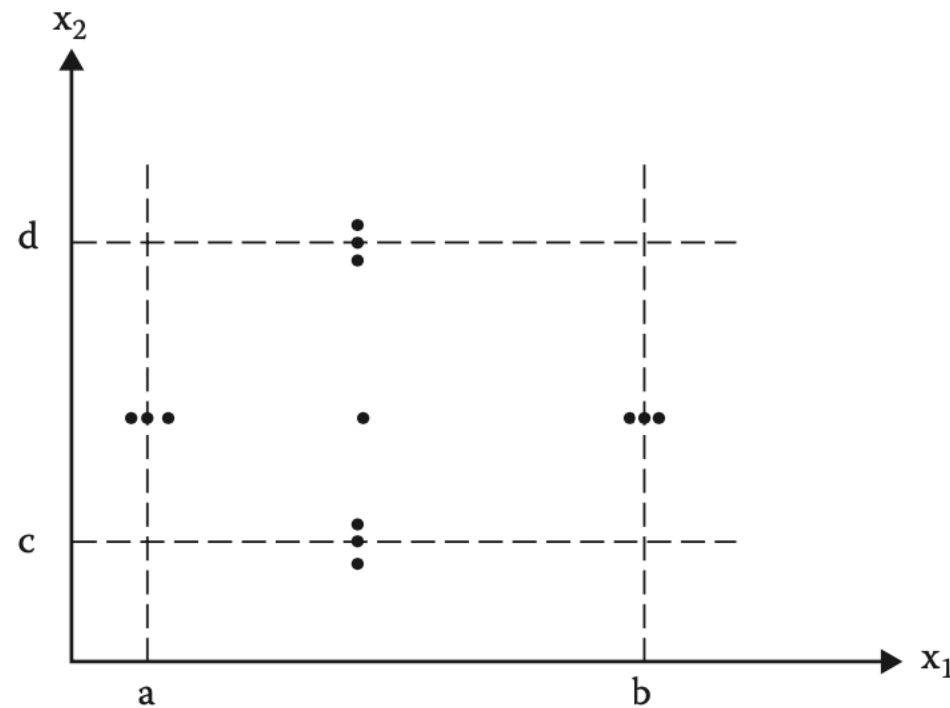
(a, mid)	(b, mid)	(mid, c)	(mid, d)
$(a+1, \text{mid})$	$(b-1, \text{mid})$	$(\text{mid}, c+1)$	$(\text{mid}, d-1)$

Robust Boundary Value Testing

For function $f(x_1, x_2)$ with domains $[a, b]$, $[c, d]$, test combinations covering:

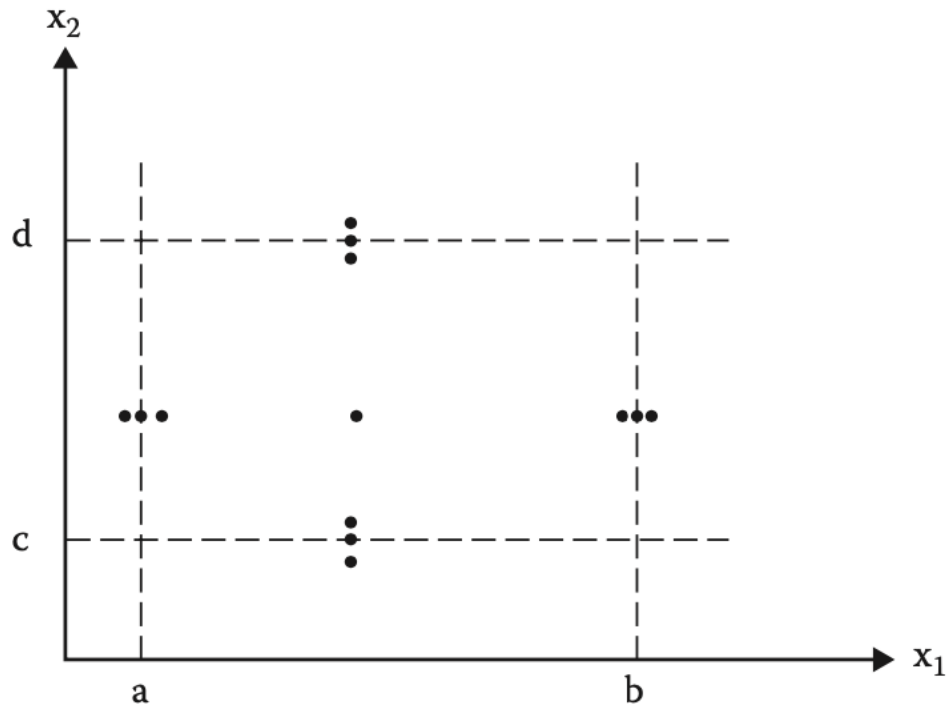
Assumptions:

- 1. assuming only valid inputs and valid type**
- 2. assuming error is caused by one variable's boundary value**



Robust Boundary Value Testing

For function $f(x_1, x_2)$ with domains $[a,b]$, $[c,d]$,
test combinations covering:



(mid, mid)



(a, mid)	(b, mid)	(mid, c)	(mid, d)
$(a+1, mid)$	$(b-1, mid)$	$(mid, c+1)$	$(mid, d-1)$
$(a-1, mid)$	$(b+1, mid)$	$(mid, c-1)$	$(mid, d+1)$

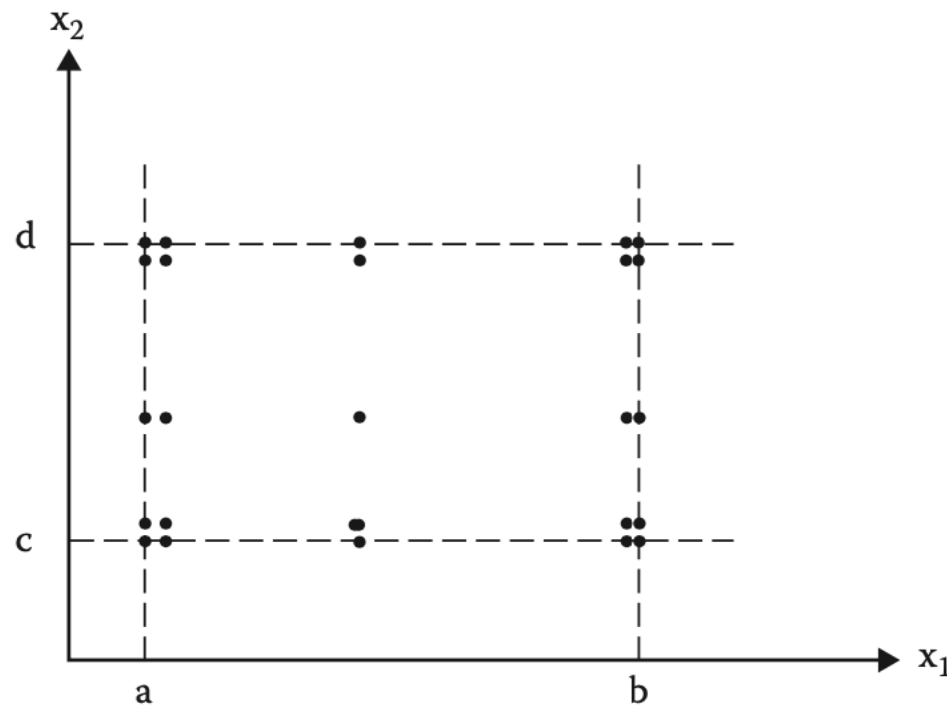
Out of range

Worst Boundary Value Testing

For function $f(x_1, x_2)$ with domains $[a, b]$, $[c, d]$, test combinations covering:

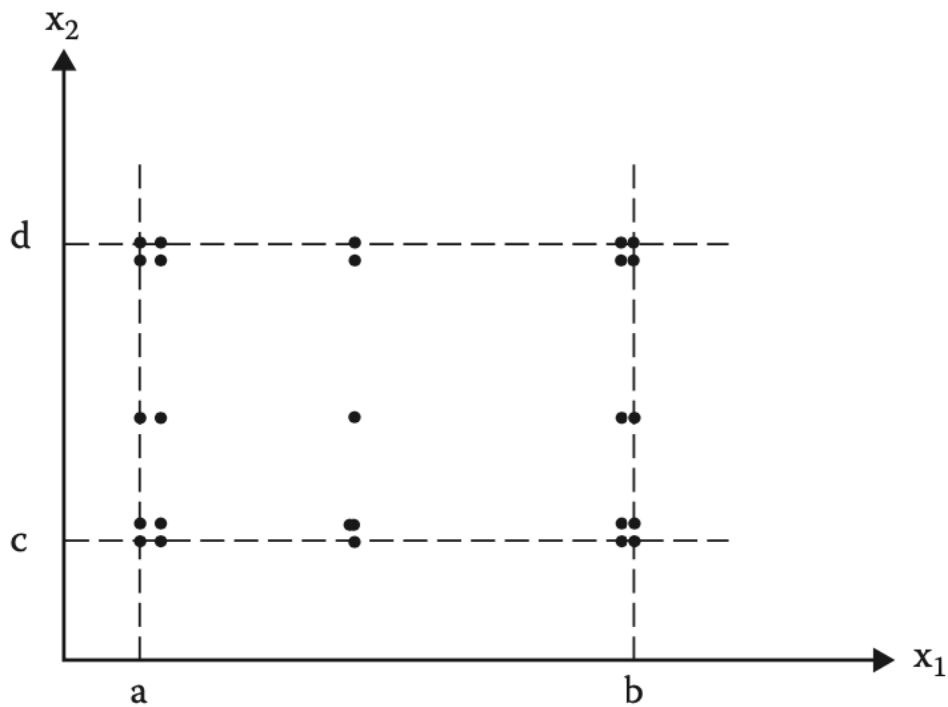
Assumptions:

- 1. assuming only valid inputs and valid type**
- 2. assuming error is caused by one variable's boundary value**



Worst Boundary Value Testing

For function $f(x_1, x_2)$ with domains $[a,b]$, $[c,d]$,
test combinations covering:



$\{a, a+1, \text{mid}, b-1, b\}$  $\{c, c+1, \text{mid}, d-1, d\}$

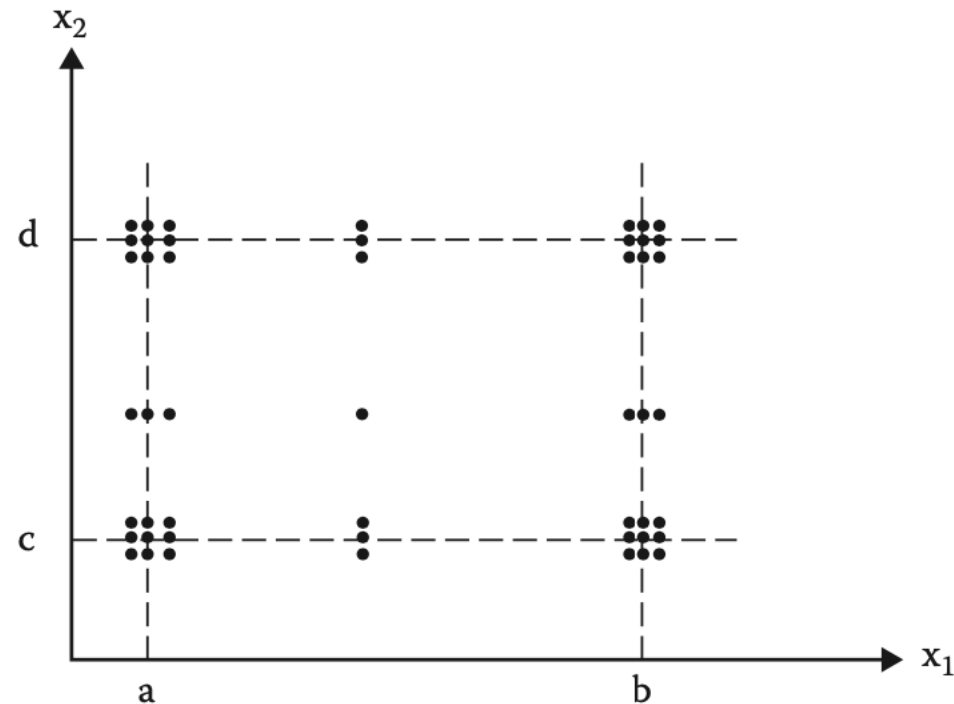
	c	c+1	mid	d-1	d
a	(a,c)	(a, c+1)	(a, mid)	(a, d-1)	(a, d)
a+1	(a+1,c)	(a+1, c+1)	(a+1, mid)	(a+1, d-1)	(a+1, d)
mid	(mid,c)	(mid, c+1)	(mid, mid)	(mid, d-1)	(mid, d)
b-1	(b-1,c)	(b-1, c+1)	(b-1, mid)	(b-1, d-1)	(b-1, d)
b	(b,c)	(b, c+1)	(b, mid)	(b, d-1)	(b, d)

Robust Worst Boundary Value Testing

For function $f(x_1, x_2)$ with domains $[a, b]$, $[c, d]$, test combinations covering:

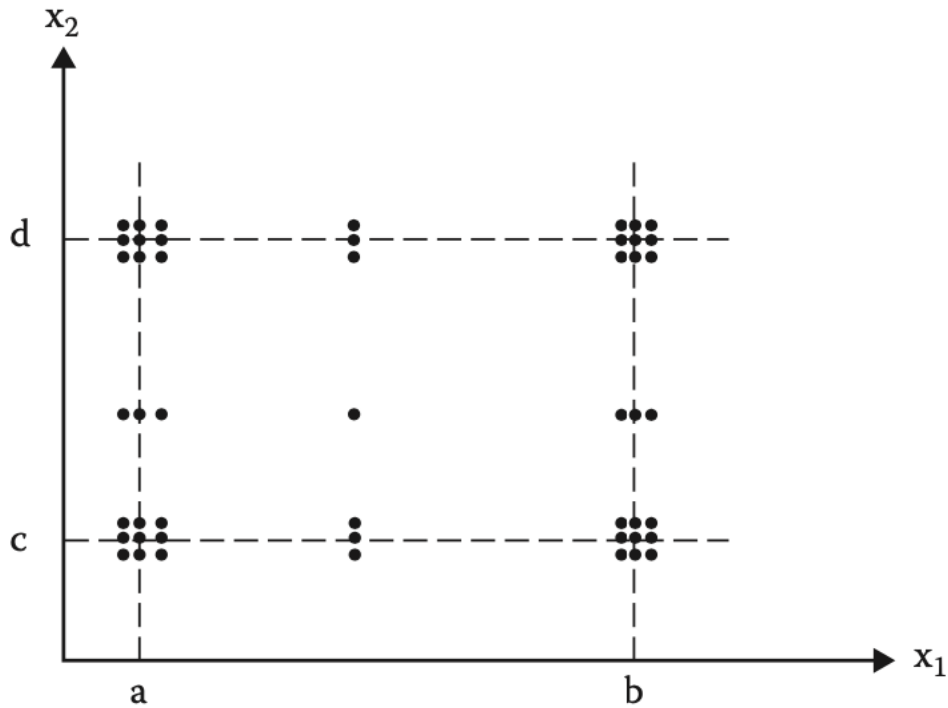
Assumptions:

- 1. assuming only valid inputs and valid type**
- 2. assuming error is caused by one variable's boundary value**



Robust Boundary Value Testing

For function $f(x_1, x_2)$ with domains $[a,b]$, $[c,d]$,
test combinations covering:



$\{\mathbf{a-1}, a, a+1, \text{mid}, b-1, b, \mathbf{b+1}\}$  $\{\mathbf{c-1}, c, c+1, \text{mid}, d-1, d, \mathbf{d+1}\}$

	c-1	c	c+1	mid	d-1	d	d+1
a-1							
a							
a+1							
mid							
b-1							
b							
b+1							

Limitations of Boundary Value Testing?



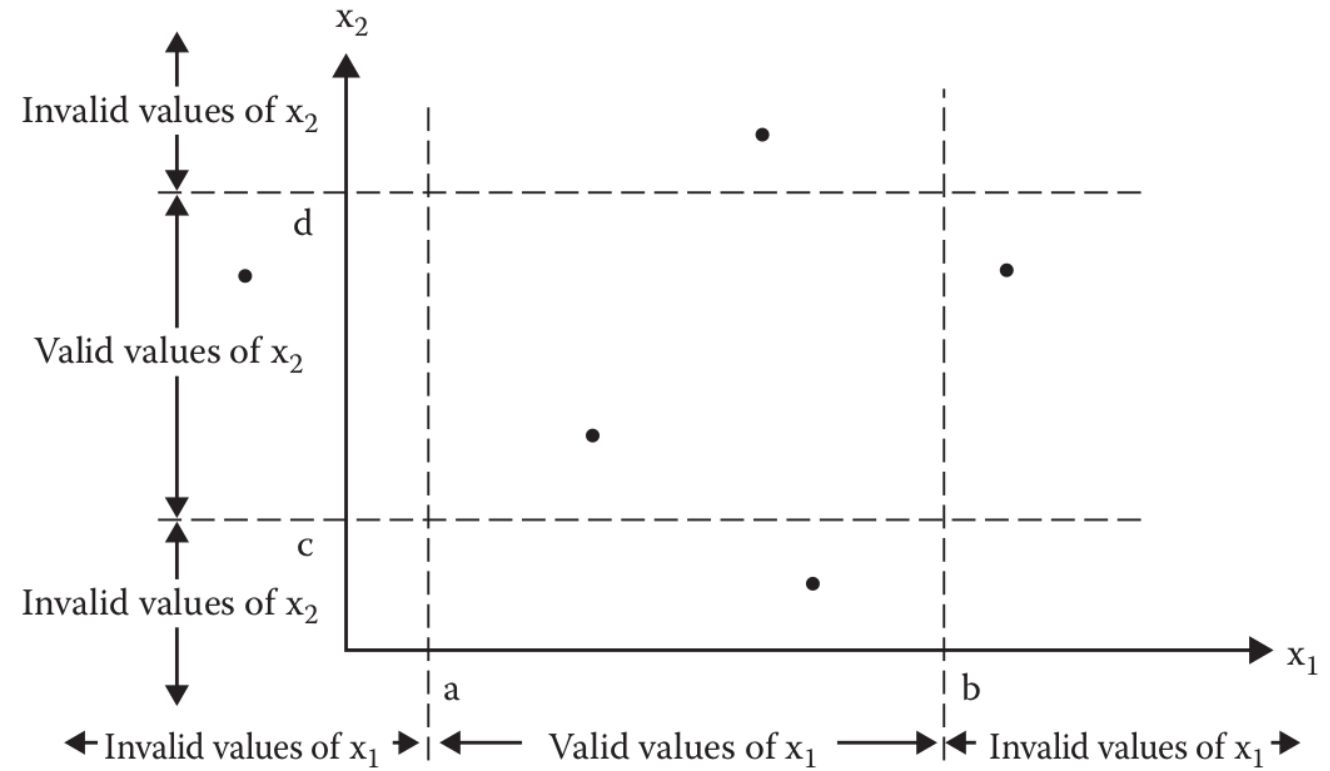
Equivalence Class Testing

For function $f(x_1, x_2)$ with domains $[a,b]$, $[c,d]$, test combinations covering:

Assumptions:

- ~~1. assuming only valid inputs and valid type~~
- ~~2. assuming error is caused by one variable's boundary value~~

One Test from one input partition



Example- Convert Time

```
1  /**
2  *
3  * @param {number} h - the current time in source timezone, in hour, in range of [0,23]
4  * @param {number} stz - the source timezone, in range of [-12, +14]
5  * timezone list https://www.ibm.com/docs/en/cloudpakw3700/2.3.0.0?topic=SS6PD2\_2.3.0/doc/psapsys\_res
6  * @param {number} ttz - the target timezone, in range of [-12, +14]
7  * @returns {[number,number]} - the current time in target timezone, in hour, in range of [0,23] and
8  function tzconvert(h, stz, ttz) {
9      // not yet implemented
10 }
```



Convert Time: Possible values for each input

$h \in [0, 23]$
Mean = 11
Min = 0
Min+1 = 1
Min-1 = -1
Max-1 = 22
Max = 23
Max+1 = 24

$stz \in [-12, +14]$
Mean = 1
Min = -12
Min+1 = -11
Min-1 = -13
Max-1 = 13
Max = 14
Max+1 = 15

$ttz \in [-12, +14]$
Mean = 1
Min = -12
Min+1 = -11
Min-1 = -13
Max-1 = 13
Max = 14
Max+1 = 15

Normal Boundary Value Testing Test Cases

Convert Time:

$h \in [0, 23]$	$stz \in [-12, +14]$	$ttz \in [-12, +14]$	Expected Output
Min=0	Mean=1	Mean=1	[0,0]
Min+1=1	Mean=1	Mean=1	[1,0]
Max-1=22	Mean=1	Mean=1	[22,0]
Max=23	Mean=1	Mean=1	[23,0]
Mean=11	Min=-12	Mean=1	[0,1]
Mean=11	Min+1= -11	Mean=1	[23,0]
Mean=11	Max-1=13	Mean=1	[23,-1]
Mean=11	Max=14	Mean=1	[22,-1]
Mean=11	Mean=1	Min=-12	[22,-1]
Mean=11	Mean=1	Min+1= -11	[23,-1]
Mean=11	Mean=1	Max-1=13	[12,0]
Mean=11	Mean=1	Max=14	[0,1]
Mean=11	Mean=1	Mean=1	[0,11]

Robust Boundary Value Testing Test Cases

$h \in [0, 23]$	$stz \in [-12, +14]$	$ttz \in [-12, +14]$	Expected Output
Min-1=-1	Mean=1	Mean=1	invalid
Min=0	Mean=1	Mean=1	[0,0]
Min+1=1	Mean=1	Mean=1	[1,0]
Max-1=22	Mean=1	Mean=1	[22,0]
Max=23	Mean=1	Mean=1	[23,0]
Max+1=24	Mean=1	Mean=1	invalid
Mean=11	Min-1= -13	Mean=1	invalid
Mean=11	Min= -12	Mean=1	[0,1]
Mean=11	Min+1= -11	Mean=1	[23,0]
Mean=11	Max-1=13	Mean=1	[23,-1]
Mean=11	Max=14	Mean=1	[22,-1]
Mean=11	Max+1=15	Mean=1	invalid
Mean=11	Mean=1	Min-1= -13	invalid
Mean=11	Mean=1	Min= -12	[22,-1]
Mean=11	Mean=1	Min+1= -11	[23,-1]
Mean=11	Mean=1	Max-1=13	[23,0]
Mean=11	Mean=1	Max=14	[0,1]
Mean=11	Mean=1	Max+1=15	invalid
Mean=11	Mean=1	Mean=1	[0,11]

Convert Time

- Worst Boundary Value Testing?
- Robust Worst Boundary Value Testing?
- Exercise
 - How many?

Convert Time – Equivalent Classes

h classes – {low_invalid, valid, high_invalid}

stz classes – {low_invalid, valid, high_invalid}

ttz classes – {low_invalid, valid, high_invalid}

Robust Boundary Value: (Assumption 2 only)

h=**low_invalid (0)**, stz=valid (1), ttz=valid (1)

h=**high_invalid (23)**, stz=valid (1), ttz=valid (1)

h=valid (11), stz=**low_valid (-12)**, ttz=valid (1)

h=valid (11), stz=**high_valid (14)**, ttz=valid (1)

h=valid (11), stz=valid (1), ttz=**low_valid (-12)**

h=valid (11), stz=valid (1), ttz=**high_valid (14)**

h=valid (11), stz=valid (1), ttz=valid (1)

Convert Time – Equivalent Classes

h classes – {low_invalid, valid, high_invalid}

stz classes – {low_invalid, valid, high_invalid}

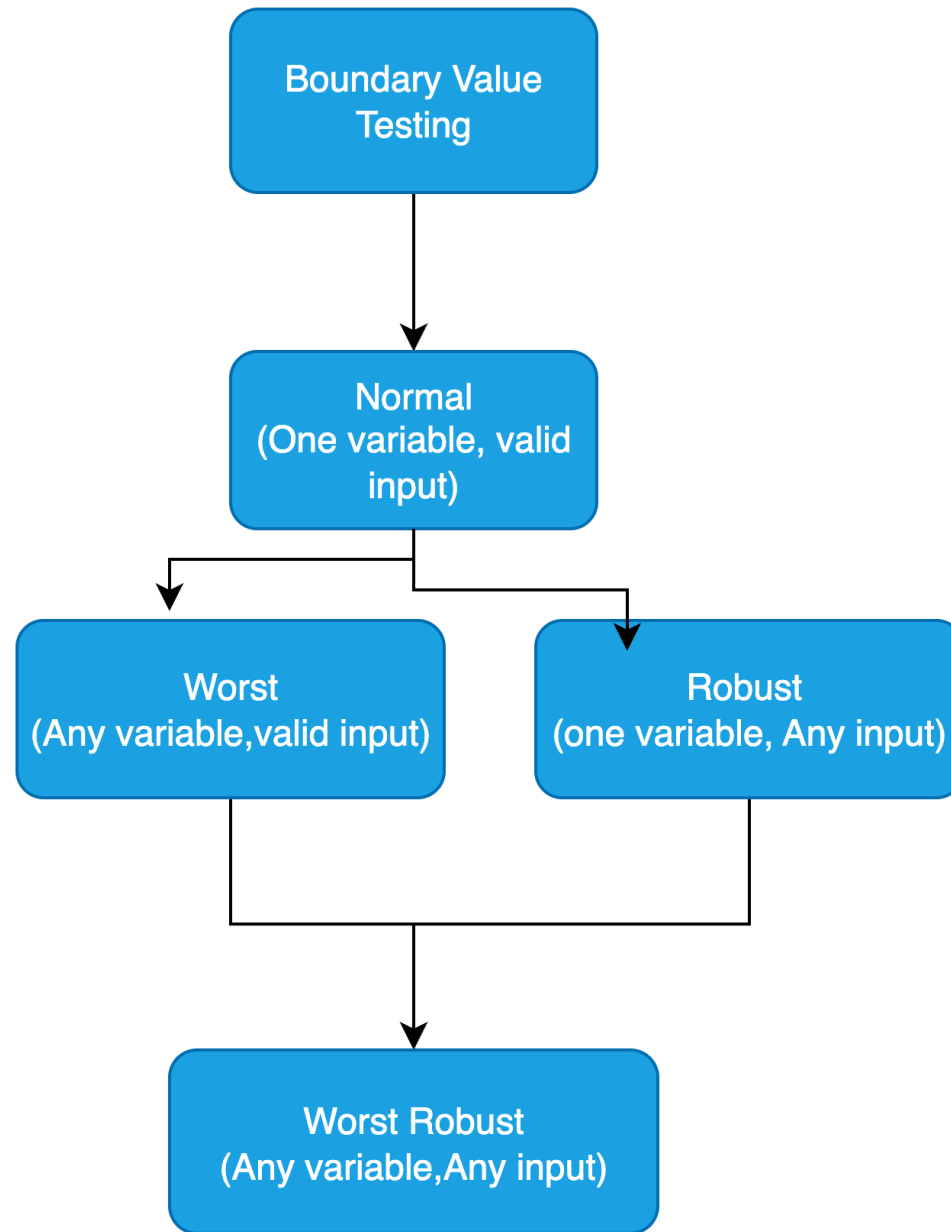
ttz classes – {low_invalid, valid, high_invalid}

Strong Robust Boundary Value: (No Assumption)

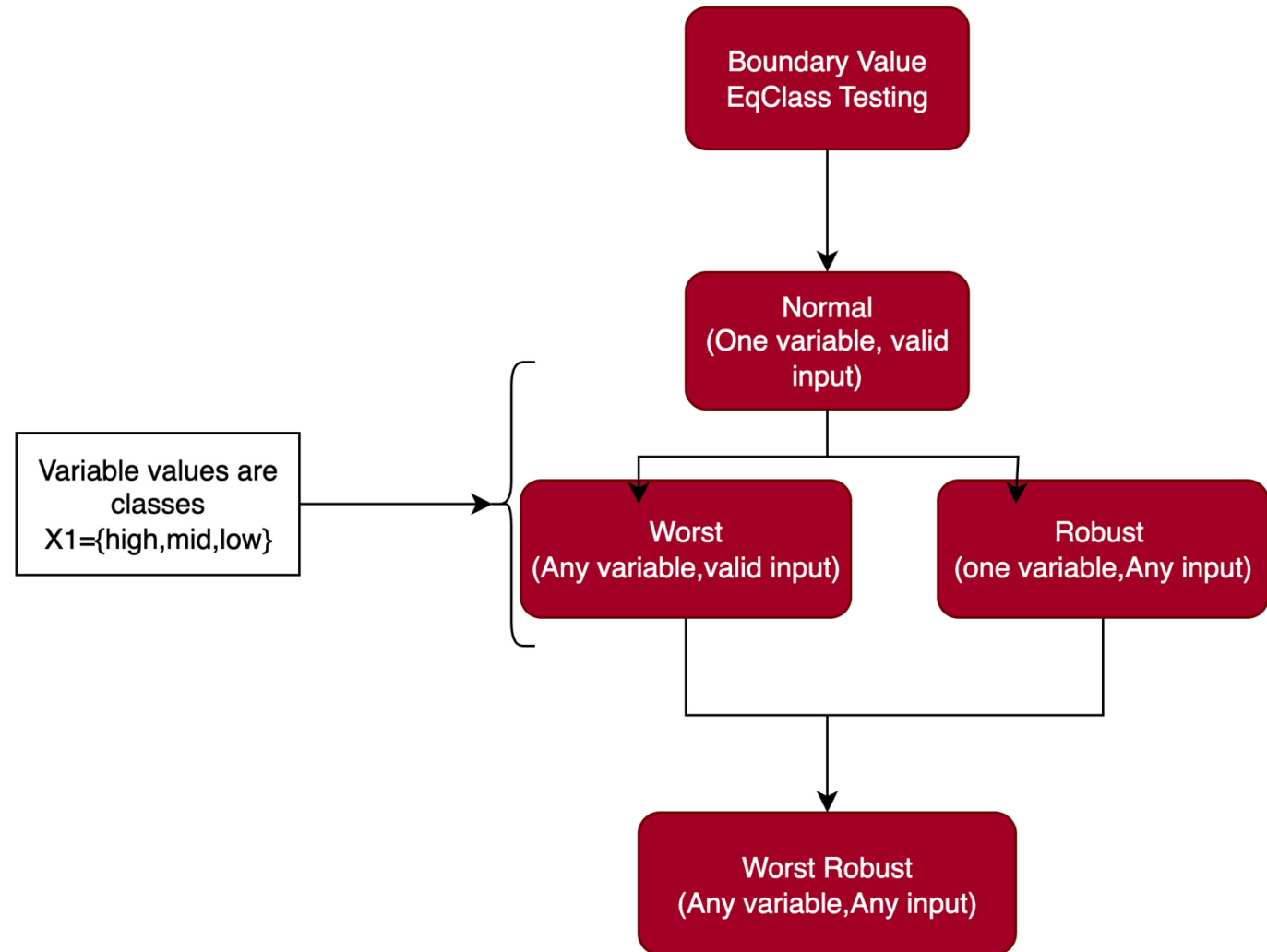
low_invalid, valid, high_invalid}  low_invalid, valid, high_invalid}  low_invalid, valid, high_invalid}

Cross Product of h classes, stz classes, ttz classes

Summary – Boundary Value Testing (Independent Variables)



Summary – Eq Class Boundary Value Testing (Independent Variables)



Decision Table Based Testing

- With x_1 , x_2 we tried,
 - (any possible x_1 , Any possible x_2)
 - x_1 , x_2 independent variables
- Some value combinations are not possible
 - day/month/year (30/02/2000) not possible
 - day, month, year dependent variables
 - Need decision table to find possible combinations

Example

```
1  /**
2   *
3   * @param {number} d - the day in digit, [1,31]
4   * @param {number} m - the month in digit, [1,12]
5   * @param {number} y - the year in digit, [0,inf)
6   * @returns {[number,number,number]} - the next day in date [day,month,year]
7   */
8  function nextdate(d,m,y) {
9      // TODO
10 }
```



Identify classes

- M1 = {month: month has 30 days}
- M2 = {month: month has 31 days except December}
- M3 = {month: month is December}
- M4 = {month: month is February}
- D1 = {day: $1 \leq \text{day} \leq 27$ }
- D2 = {day: day = 28}
- D3 = {day: day = 29}
- D4 = {day: day = 30}
- D5 = {day: day = 31}
- Y1 = {year: year is a leap year}
- Y2 = {year: year is a common year}

Decision Table (1)

~	1	2	3	4	5	6
m	M1	M1	M1	M1	M1	M2
d	D1	D2	D3	D4	D5	D1
y	-	-	-	-	-	-
actions						
impossible					X	
d+1	X	X	X			X
d=1				X		
m+1				X		
m=1						
y+1						

Decision Table (2)

~	11	12	13	14	15	16
m	M3	M3	M3	M3	M3	M4
d	D1	D2	D3	D4	D5	D1
y	-	-	-	-	-	-
actions						
impossible						
d+1	X	X	X	X		X
d=1					X	
m+1						
m=1					X	
y+1					X	